

Project Handover Documentation - Dantalabs Pipeline V2

To: Dani
From: Antigravity
Date: 2026-01-26

1. Project Overview

Dantalabs Pipeline V2 is a lead generation and enrichment platform. It helps users find potential business leads (e.g., "Marketing Agencies in Hanoi") by scanning Google Maps, qualifying them based on specific criteria (e.g., website speed, tech stack), and enriching the data with contact info.

2. Tech Stack

Frontend

- Framework: [Next.js 15+](#) (App Directory)
- Language: TypeScript
- Styling: Tailwind CSS, Lucide React (Icons), Shadcn UI (Components)
- Key Libraries: [@tanstack/react-query](#) (Data Fetching), [sonner](#) (Toasts).

Backend

- Framework: [FastAPI](#) (Python)
- Server: Uvicorn
- Key Libraries:
 - [apify-client](#): For running Google Maps scrapers.
 - [google-generative-ai](#): Google Gemini for query optimization and intelligence.
 - [supabase](#): Database interactions.
 - [beautifulsoup4](#): HTML parsing/scraping.

Infrastructure & Services

- Database: Supabase (PostgreSQL)
- AI Model: Google Gemini (Flash 1.5/2.0)
- Scraper: Apify (Actor: [compass/crawler-google-places](#))

3. Core Architecture

The app follows a client-server architecture:

1. Next.js Frontend: Handles UI (Scan Dialog, Dashboard). Sends requests to Next.js API Routes (acting as a proxy) or directly to the Python Backend.
2. Python Backend: Exposes REST endpoints (e.g., `/scan`).

- Step 1: Optimization: Uses Gemini to refine user keywords (e.g., "Coffee" -> "Specialty Coffee Shop").
- Step 2: Scanning: Calls Apify to scrape Google Maps results.
- Step 3: Qualification: Checks website PageSpeed, Tech Stack (Detector), and other criteria to qualify leads.
- Step 4: Storage: Saves qualified leads to Supabase.

4. Key Features

Lead Scanning

- Dynamic Location: Users can search in ANY location (e.g., "New York", "Hanoi").
- Strict Filtering: If a location is specified, results are treated strictly to ensure accuracy. If strict search fails, it falls back to a broader search.
- Smart Optimization: Gemini AI optimizes search terms for better results on Google Maps.

Qualification Logic

Leads are "QUALIFIED" if they meet certain criteria indicating they might need services (e.g., Slow Website, No SSL, using WordPress).

5. Setup & Installation

Prerequisites

- Python 3.9+
- Node.js 18+

Environment Variables (.env)

You need the following keys:

```
SUPABASE_URL=...
SUPABASE_KEY=...
GOOGLE_API_KEY=...
APIFY_API_TOKEN=...
```

Running Locally

1. Backend

Navigate to root

```
python -m venv .venv
```

```
source .venv/bin/activate # or .venv\Scripts\activate on Windows
```

```
pip install -r requirements.txt
```

Run server

```
uvicorn api.index:app --reload
```

(Note: Check

api/index.py or

backend/main.py for exact entry point)

2. Frontend

npm install

npm run dev

Visit <http://localhost:3000>.

6. Recent Updates (Important for Dani)

- Dynamic Location Scanning: I just updated scan.py and scan-dialog.tsx.
 - Previously, location was hardcoded to "Ho Chi Minh City".
 - Now, there is a Location Input in the UI.
 - Logic:
 - If Location provided -> Strict filter applied.
 - If Location empty -> Global search (no filter).

7. Known Issues / Notes

- PageSpeed API: Sometimes fails if the target site blocks bot requests. Handled gracefully in
- scan.py.
- Apify Costs: Large scans consume Apify compute units. Watch the usage.

8. File Structure Highlights

- src/components/features/pipeline/ -> UI Components for the dashboard.
- backend/routers/ -> API Endpoints (scan.py, draft.py).
- backend/services/ -> Business logic (Gemini wrapper, Scraper utils).